

FORMATION INTENSIVE · 3 JOURS · 100% PRATIQUE

DevOps pratique avec Docker et CI/CD



« De l'application au déploiement : Docker, CI/CD et bonnes pratiques DevOps »

Code → Git → Docker → Pipeline CI/CD → Serveur → Application déployée

Formateur : Mouhamoud DEMBELE du 09/09/2026 au 11/09/2026

Et vous, qui êtes-vous ?

Chacun se présente en 1 minute, en suivant ce canevas :

1



Nom & parcours

Ton prénom, ta filière et ton niveau d'études

2



Ce que je connais déjà

Les langages / outils que tu maîtrises un peu

3



Ce que j'attends

Ce que tu espères savoir faire à la fin des 3 jours

4



Un fun fact

Une info insolite sur toi, pour qu'on se souvienne de toi !

QUI SUIS-JE ?

Présentation du formateur



NOM & PRÉNOM

DEMBELE Mouhamoud

POSTE / FONCTION

Ingénieur Developpeur / DevOps /Formateur Web/Mobile

EXPÉRIENCE

[14 années en développement / DevOps]

COMPÉTENCES CLÉS

Java, Js, Python, Docker, CI/CD, Cloud...

CONTACT

Le programme des 3 jours



JOUR 1

Docker & conteneurisation

- ✓ Images, conteneurs, volumes, réseaux
- ✓ Dockerfile & Docker Compose
- ✓ Conteneuriser une application complète



JOUR 2

Intégration continue (CI)

- ✓ Git & stratégie de branches
- ✓ Premier pipeline GitLab CI/CD
- ✓ Build, tests et image automatisés



JOUR 3

Déploiement (CD) & bonnes pratiques

- ✓ Déploiement automatique en ligne
- ✓ Secrets, HTTPS, logs & monitoring
- ✓ Mini-projet de bout en bout

Objectifs & informations pratiques

À l'issue de la formation, vous saurez :

- ✓ Conteneuriser une application avec Docker & Docker Compose
- ✓ Mettre en place un pipeline CI/CD avec Git et GitLab CI/CD
- ✓ Déployer une application en ligne, gratuitement et en HTTPS
- ✓ Gérer secrets, logs et supervision comme en entreprise

LOGISTIQUE



9h – 16h

avec 1h de pause (12h30 – 13h30)



100% pratique

chaque notion suivie d'un atelier immédiat



Ordinateur portable

obligatoire, avec Docker installé



Outils 100% gratuits

aucun frais pour suivre les TP



JOUR 1 · MATIN

Docker & conteneurisation

Comprendre les conteneurs et emballer une application

« Ça marche sur ma machine... »

Sans conteneur



Versions différentes entre postes



Dépendances manquantes en prod



« Ça marche pas chez toi »

Avec Docker



Le même environnement partout



Application + dépendances embarquées



Déploiement rapide et fiable

Les 4 concepts clés de Docker



Image

Le « plan » figé de l'application :
code + dépendances +
configuration



Conteneur

Une instance en cours d'exécution
d'une image, isolée et légère



Volume

Un espace de stockage persistant,
indépendant du cycle de vie du
conteneur



Réseau

Un canal de communication entre
conteneurs (ex : API ↔ base de
données)

Dockerfile & Docker Compose

Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

docker-compose.yml

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    env_file: .env
    depends_on:
      - db
  db:
    image: mongo:7
```

```
docker run --name OSC_nginx -p 8082:80 nginx
```

Mouhamoud.dembele@gmail.com



À vous de jouer !

- 1 Installer Docker et vérifier avec docker run hello-world
- 2 Écrire un Dockerfile pour votre application (ou l'app fournie)
- 3 Construire l'image et lancer le conteneur (build, run, ports)
- 4 Ajouter volumes et variables d'environnement (.env)
- 5 Orchestrer API + base de données avec Docker Compose

 *Détail complet des ateliers : Fiche de TP*



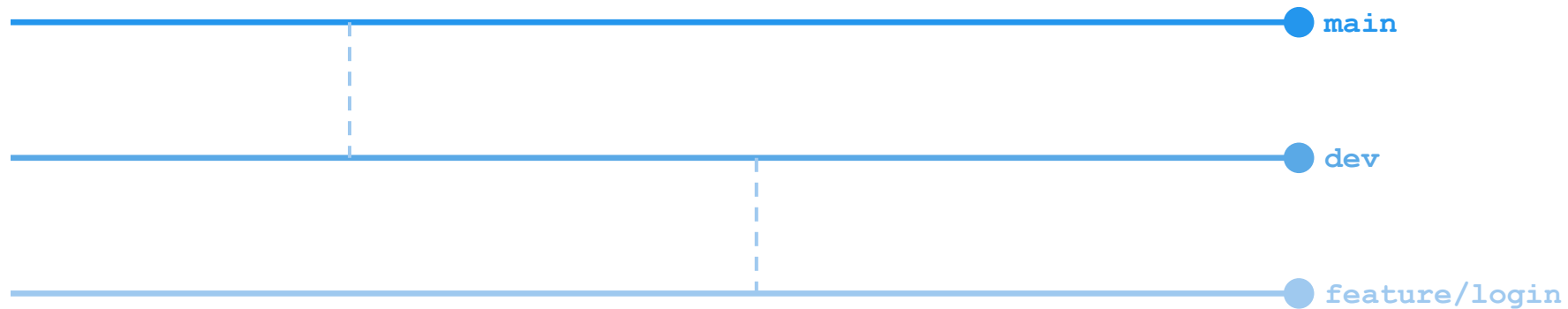
JOUR 2 · MATIN

Intégration continue (CI)

Automatiser le build, les tests et la création de l'image

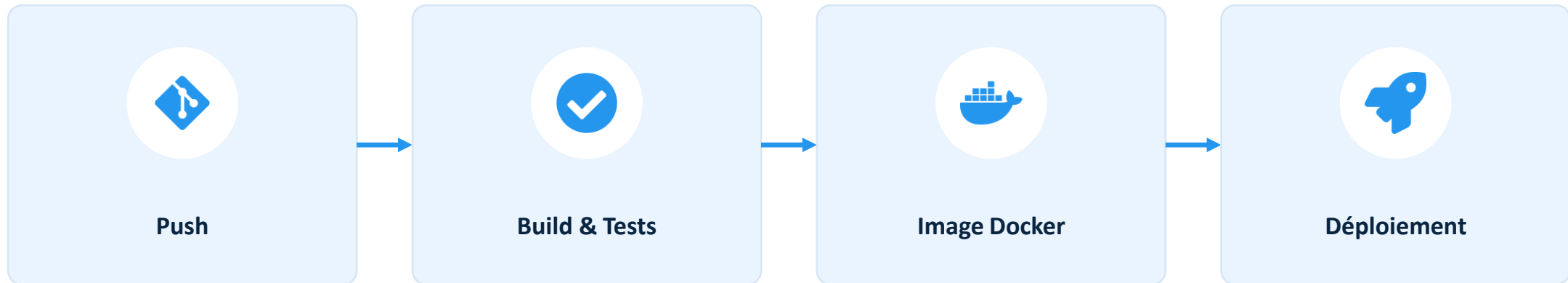
Git & stratégie de branches

Schéma simplifié :



- ✓ main : toujours stable, prête pour la production
- ✓ dev : intégration des fonctionnalités en cours
- ✓ feature/* : une branche par fonctionnalité, fusionnée par Pull Request

Le pipeline CI/CD



Intégration continue (CI) : chaque push déclenche build + tests automatiques.

Livraison continu (CD) : si tout est vert, l'application est déployée automatiquement.

Le paysage des outils CI/CD en entreprise

GitLab CI/CD n'est pas le seul outil du marché — voici les principaux, pour situer ce qu'on pratique :

NOTRE CHOIX



GitLab CI/CD

SaaS ou auto-hébergé · Open source

Notre choix pour ce TP : repos privés gratuits, tout intégré (code + CI + registre d'images)



GitHub Actions

SaaS · Propriétaire (Microsoft)

Très répandu, énorme marketplace d'actions prêtes à l'emploi



Jenkins

Auto-hébergé · Open source

Le standard historique en entreprise, très flexible mais demande un serveur à maintenir



CircleCI & autres

SaaS · Propriétaire

Alternatives (CircleCI, Azure DevOps, Bitbucket Pipelines...) selon l'écosystème cloud

Anatomie d'un pipeline GitLab CI/CD

`.gitlab-ci.yml`

```
stages:
  - test
  - build

run_tests:
  stage: test
  image: node:20
  script:
    - npm install
    - npm test

build_image:
  stage: build
  script:
    - docker build -t api .
```

stages

L'ordre des grandes étapes du pipeline (test, build, deploy...)

image

L'environnement Docker utilisé pour exécuter le job

script

La suite de commandes exécutées dans le job

Chaque push déclenche le pipeline automatiquement — visible dans l'onglet CI/CD ▶ Pipelines de votre projet GitLab.



À vous de jouer !

- 1 Créer un compte GitLab personnel et initialiser un dépôt privé
- 2 Pousser le projet avec une stratégie de branches
- 3 Écrire un premier pipeline GitLab CI/CD (.gitlab-ci.yml)
- 4 Ajouter la construction de l'image Docker au pipeline
- 5 Vérifier l'exécution automatique dans l'onglet CI/CD ► Pipelines

 *Détail complet des ateliers : Fiche de TP*

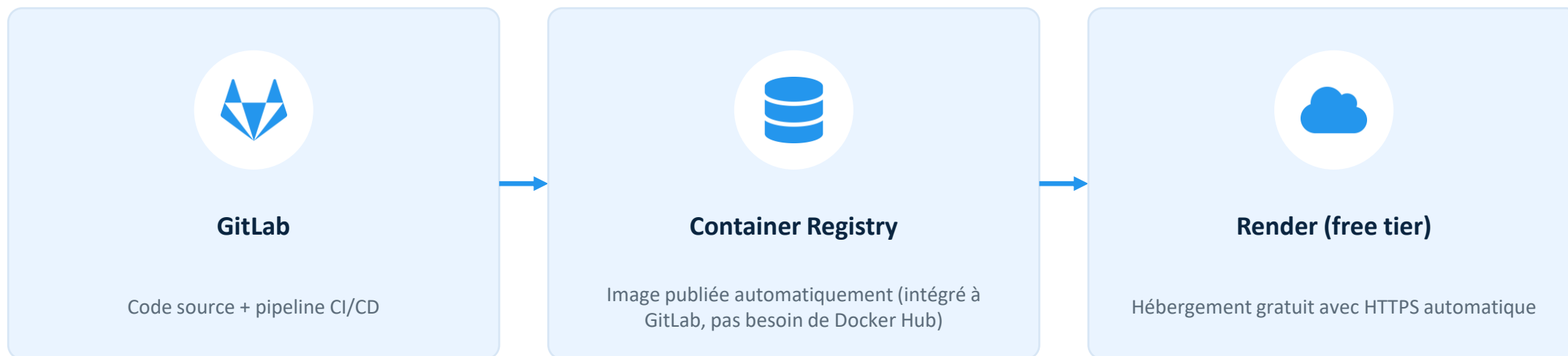


JOUR 3 · MATIN

Déploiement & bonnes pratiques

Mettre son application en ligne, gratuitement et en sécurité

De l'image Docker à l'application en ligne



100% gratuit : Render (ou Railway) offre un plan gratuit avec URL publique et HTTPS automatique — aucun serveur à payer pour le TP.

Sécurité, secrets & supervision



Secrets

Variables d'environnement stockées côté hébergeur, jamais dans Git



HTTPS

Chiffrement automatique des échanges (certificat géré par la plateforme)



Logs & monitoring

Suivre l'activité et détecter les erreurs en production



Mini-projet de déploiement

- 1 Créer un compte gratuit sur Render et connecter le dépôt GitLab
- 2 Déployer l'image Docker : obtenir une URL publique en HTTPS
- 3 Configurer les secrets (clé JWT, URI base de données...)
- 4 Étendre le pipeline pour un déploiement automatique à chaque push
- 5 Vérifier les logs, puis présenter son application déployée

 *Détail complet des ateliers : Fiche de TP*



Bravo, votre application est en ligne !

Code → Git → Docker → Pipeline CI/CD → Serveur → Application déployée

Merci et bravo pour ces 3 jours ! Des questions ?